Σχεδιασμός και Υλοποίηση All-in-one Controller με LCD Screen και Ενσωμάτωση AI

Όνομα Ομάδας: Τα παιδιά από την Πάτρα

Μέλη:

Γεώργιος Σικαλιάς

Ιάσονας Στρατουδάκης

Κωσταντίνος Παπαδημητρίου

ABSTRACT

Η συγκεκριμένη εργασία ασχολείται με τη σχεδίαση και την υλοποίηση μιας ολοκληρωμένης συσκευής All-in-one Controller, στο πλαίσιο του διαγωνισμού PreΣΦΗΜΜΥ 10 του συνεδρίου ΣΦΗΜΜΥ 17. Η συσκευή έχει ως πυρήνα τον microcontroller ESP32 DevKit V1 σε συνδυασμό με μία οθόνη TFT LCD 3.5" eSPI και έναν I/O Expander 16-bit MCP 23017 πρωτοκόλλου I2C. Η συσκευή επιτρέπει τον έλεγχο του λειτουργικού συστήματος και γενικώς των multimedia, shortcuts για εφαρμογές όπως Spotify, Zoom και Discord αλλά και για το λογισμικό streaming Streamlabs OBS, καθώς και real-time visual response και feedback μέσω της οθόνης. Τα μέσα αλληλεπίδρασης συσκευής-χρήστη περιλαμβάνουν: 13 διακόπτες τύπου button, ένα rotary encoder και διεπαφή φωνητικών εντολών με επεξεργασία φυσικής γλώσσας (NLP) που αξιοποιεί το Gemini AI για βελτιστοποίηση της κατανόησης των εντολών. Ο προγραμματισμός του microcontroller και της οθόνης είναι σε Arduino C++ με τη διασύνδεση του με το λειτουργικό σύστημα και τις εφαρμογές στο PC να έχουν υλοποιηθεί σε Python. Η συσκευή αυτή είναι τοποθετημένη σε ένα 3D printed fully modular κουτί σχεδιασμένο σε περιβάλλον CAD, το οποίο εκτός από την πρακτικότητα του στην συναρμολόγηση-αποσυναρμολόγηση προσδίδει σύγχρονη αισθητική στη συσκευή. Ο συνολικός προϋπολογισμός της συσκευής ανέρχεται στα 61.34€.

CCS CONCEPTS • Computer systems organization ~ Embedded systems • Human-centered computing ~ Interaction devices • Computing methodologies ~ Artificial intelligence

Additional Keywords and Phrases: ESP32, MCP23017, Gemini AI, Google LLM, Διασύνδεση Ανθρώπου-Μηχανής (HCI), Python Serial Bridge, Έλεγχος Πολυμέσων, LCD Screen GUI, Real-time Visual Feedback.

ACM Reference Format:

Γεώργιος Σικαλιάς, Ιάσονας Στρατουδάκης, Κωσταντίνος Παπαδημητρίου. 2026. Σχεδιασμός και Υλοποίηση All-in-one Controller με LCD Screen και Ενσωμάτωση AI. PreΣΦΗΜΜΥ 10, ΣΦΗΜΜΥ 17. Πάτρα, Ελλάδα.

Contents

1 INTRODUCTION 4

1.1 Concept 4

1.2 Επιλογή hardware 4

1.3 Σχεδίαση Software 5

1.4 Hardware Συσκευής και Κοστολόγηση 5

2 HARDWARE DESIGN 6

2.1 Τροφοδοσία και Ισχύς 6

2.2 Διεπαφές Επικοινωνίας 6

2.2.1 I2C Bus 6

2.2.2 SPI Bus 7

2.3 Διασύνδεση Εισόδων 8

2.3.1 Μηχανικοί Διακόπτες (Buttons) 8

2.3.2 Περιστροφικός Κωδικοποιητής (Rotary Encoder) 8

2.3.3 Breadboards 9

3 C++ FIRMWARE & SERIAL BRIDGE 9

3.1 Δομή Firmware 9

3.2 Διαχείριση εισόδων 9

3.3 Σχεδίαση GUI Οθόνης 9

3.4 Σειριακό Πρωτόκολλο Επικοινωνίας 10

3.5 Λογική Serial Bridge 10

4 PYTHON SOFTWARE & API'S 10

4.1 Διαχείριση Λειτουργικού Συστήματος 10

4.2 Διαχείριση Εφαρμογών 11

4.2.1 Spotify 11

4.2.2 Discord (Καθολικός Έλεγχος Κώφωσης - Deafen) 11

4.2.3 Zoom (Raise Hand Integration) 11

4.2.4 OBS (Έλεγχος μέσω OS-Level Hotkeys) 12

4.3 Διεπαφή Χρήστη στον υπολογιστή (PC GUI) 12

4.3.1 Έξυπνη Ανίχνευση και Εκκίνηση Εφαρμογών 12
4.3.2 Δυναμική Βαθμονόμηση Μικροφώνου (Microphone Calibration) 12
4.4 Δυναμικά Προσαρμόσιμα Πλήκτρα (Custom Buttons) 13
4.5 Ηχογράφηση και Επεξεργασία Φωνής 13
4.5.1 Ηχογράφηση Φωνής 13
4.5.2 Επεξεργασία Φωνής 13
4.5.3 Speech-to-text μέσω της Google 13
4.6 Απόκριση σε Φωνητικές Εντολές και Ενσωμάτωση AI 13
4.6.1 Local Parsing 14
4.6.2 AI Processing 14
5 3D PRINTING & DESIGN 14
5.1 Φιλοσοφία και Διαδικασία Σχεδίασης 14
5.2 Ανατομία Περιβλήματος 15
5.3 Ανοχές και Διορθώσεις κατά τη Συναρμολόγηση 18
6 CONCLUSION 18
7 REFERENCES – LINKS 19
7.1 Πηγές 19
7.2 Σύνδεσμοι 19

1 INTRODUCTION

1.1 Concept

Με τη συμμετοχή μας στο διαγωνισμό PreΣΦΗΜΜΥ 10 αποφασίσαμε να ασχοληθούμε με τις 3 από τις 4 αναγραφόμενες στην εκφώνηση λειτουργίες (τουλάχιστον 2 υποχρεωτικές):

- PC & Multimedia Control
- Streaming & Production
- Visual Feedback

Η επιλογή αυτών των λειτουργιών έγινε βάσει του μεγάλου impact που έχει στην καθημερινή ψηφιακή ζωή του μηχανικού η απλοποίηση και αυτοματοποίηση των διαδικασιών αυτών. Είναι πράγματι πολύ πρακτικός ο έλεγχος του λειτουργικού συστήματος και των εφαρμογών του μέσω ενός εξωτερικού συγκεντρωμένου All-in-one Controller. Επομένως, θέσαμε ως προτεραιότητα τη σχεδίαση και υλοποίηση των λειτουργιών αυτών με προτεραιότητα την αύξηση της παραγωγικότητας του χρήστη. Φυσικά, λάβαμε υπόψιν μας ως βασικό άξονα το οικονομικό κόστος το οποίο επιβαλλόταν να ελαχιστοποιηθεί, χωρίς όμως να υποβαθμιστεί η συνολική ποιότητα και λειτουργικότητα της συσκευής. Έτσι, δημιουργήσαμε από την αρχή μία value for money ανταγωνιστική συσκευή, η οποία ενσωματώνει ένα μεγάλο πλήθος διεργασιών.

1.2 Επιλογή hardware

Ως πυρήνας της συσκευής επιλέχθηκε ο microcontroller ESP32 DevKit V1 επειδή συνδυάζει υψηλή υπολογιστική ισχύ (Dual Core, Clock Speed: 240 MHz, SRAM: 520 KB, Flash Memory: 4 MB) και εξαιρετικά χαμηλό κόστος (Ενότητα 1.3). Καθώς όμως παρά το μεγάλο πλήθος των pins του ESP32 γρήγορα συνειδητοποιήσαμε ότι αυτά δεν επαρκούσαν, κατευθυνθήκαμε προς την ενσωμάτωση ενός I/O Expander και συγκεκριμένα του MCP 23017 με πρωτόκολλο I2C. Η επιλογή του MCP23017 δεν έγινε μόνο για την εξοικονόμηση pins, αλλά και για τη μείωση της πολυπλοκότητας της καλωδίωσης. Μέσω του διαύλου I2C (Serial Data - SDA, Serial Clock - SCL), ο ESP32 διαχειρίζεται το σύνολο των 13 εισόδων χρησιμοποιώντας μόνο 2 pins, αφήνοντας τους υπόλοιπους πόρους ελεύθερους για την απαιτητική οδήγηση της TFT οθόνης μέσω του πρωτοκόλλου SPI.

Παράλληλα, επειδή θεωρήσαμε πολύ σημαντική την οπτική απεικόνιση, επενδύσαμε ένα σημαντικό μέρος του συνολικού προϋπολογισμού σε μία ποιοτική ευμεγέθη οθόνη 3.5”, ώστε οι πληροφορίες να είναι διακριτές και ευανάγνωστες στο πλαίσιο ενός απλού και user-friendly GUI. Με αυτό τον τρόπο, από ένα απλό panel κουμπιών, η συσκευή απέκτησε τη δυνατότητα να προσφέρει live visual feedback με ανανέωση ανά δευτερόλεπτο για λόγους ακρίβειας και αξιοπιστίας.

1.3 Σχεδίαση Software

Καταρχάς, για τα βασικά Multimedia θα έπρεπε να αξιοποιήσουμε τα δεδομένα του λειτουργικού συστήματος (ένταση ήχου, ένταση μικροφώνου, κάμερα, χρήση CPU, RAM GPU). Στη συνέχεια, αποφασίσαμε ότι θα ενσωματώσουμε τις παρακάτω εφαρμογές:

- Spotify
- Discord
- Zoom
- Streamlabs OBS

Για τη διασύνδεση όλων αυτών των εφαρμογών θα έπρεπε να υλοποιήσουμε μία serial bridge, η οποία να επικοινωνεί με τον ESP32 μέσω USB σε micro USB καλωδίου.

Για λόγους πρακτικότητας επιλέξαμε να υλοποιήσουμε τα παραπάνω σε Python στο Visual Studio Code IDE καθώς εκτελούνται αποκλειστικά στο PC, χωρίς όμως αυτό να μας δημιουργήσει προβλήματα με τον χρόνο απόκρισης. Ο κώδικας του ESP32 υλοποιήθηκε σε Arduino C++ στο Arduino IDE και περιλαμβάνει όλες τις λειτουργίες που επιλέξαμε καθώς και δυνατότητα ενσύρματης επικοινωνίας με τον υπολογιστή μέσω του serial bridge.

1.4 Hardware Συσκευής και Κοστολόγηση

No.	Όνομα Component	Link	Κόστος
1	Espressif ESP32D Development Kit Wifi & Bluetooth	https://www.skroutz.gr/s/37404120/Espressif-ESP32D-Development-Kit-WiFi-Bluetooth.html	9.20€
1	Οθόνη LCD 3.5” (46751)	https://www.skroutz.gr/s/46180029/Othoni-LCD-3-5-46751.html	25.00€
1	Microchip I/O Expander (MCP23017-E/SP)	https://www.skroutz.gr/s/45257010/Microchip-I-O-Expander-MCP23017-E-SP.html	4.50€
1	Rotary Encoder Module with Demo Code Module για Arduino KY-040	https://www.skroutz.gr/s/40170309/Rotary-Encoder-Module-With-Demo-Code-Module-gia-Arduino-KY-040.html	2.00€
2	Haitronic Διακόπτης Μπουτόν 10τμχ	https://www.skroutz.gr/s/52173703/Haitronic-Diakoptis-Mpouton-HR0320-10tmch.html	1.00€
2	Optum Solderless Breadboard (MB-102)	https://www.skroutz.gr/s/30576026/Optum-Solderless-Breadboard-MB-102.html	4.64€
1	Haitronic Half-Size Solderless Breadboard 400 θέσεων 82x54mm	https://www.skroutz.gr/s/28640210/Haitronic-Half-Size-Solderless-Breadboard-400-theseon-82x54mm.html	1.99€
3	Dupont M-F Καλώδιο 0.2m OEM478 10τμχ	https://www.skroutz.gr/s/64645470/dupont-m-f-kalodio-0-2m-oem478-10tmch.html	2.25€
45/65	Arduino 65pcs Jumper Wire Cable Kit for Solderless Breadboard Connections	https://www.skroutz.gr/s/28756792/Arduino-65pcs-Jumper-Wire-Cable-kit-for-Solderless-Breadboard-Connections.html	2.17€

14	Πυκνωτής Κεραμικός 104 100nf	https://www.skroutz.gr/s/58711482/Pyknotis-Keramikos-104-100nf.html	0.70€
0.435kg	Esun PLA 3D Printer Filament 1.75mm Μαύρο 1kg 1τμχ (Spool)	https://www.skroutz.gr/s/59842404/esun-pla-3d-printer-filament-1-75mm-mayro-1kg-1tmch-spool.html	6.92€
1	Ρεύμα/Φθορές 3D Printer	-	1.00€
1	Λογισμικό	-	0.00€
			61.34€

2 HARDWARE DESIGN

2.1 Τροφοδοσία και Ισχύς

Η κύρια τροφοδοσία (5V) γίνεται μέσω του καλωδίου USB σε micro USB από το PC στο ESP32. Το ESP32 εν συνεχεία λειτουργεί ως κόμβος τροφοδοσίας, αφού όλα τα υπόλοιπα components (LCD screen, I/O expander, rotary controller) συνδέονται σε pins του, είτε άμεσα, είτε έμμεσα μέσω breadboard. Επειδή όμως τα υπόλοιπα στοιχεία λειτουργούν στα 3.3V, το ESP32 υποβιβάζει τη τάση στα 3.3V (μέσω LDO AMS1117) και οι συνδέσεις με τα components γίνονται μέσω του pin 3V3.

Για να φιλτράρουμε τον ηλεκτρικό θόρυβο που προκαλείται από τις εναλλαγές του σήματος(από λογικό ‘0’ σε λογικό ‘1’ και αντίστροφα) χρησιμοποιήσαμε κεραμικούς πυκνωτές των 100nF στους διακόπτες τύπου button. Έτσι, αποφεύγεται το bouncing των buttons κατά το press και διασφαλίζεται η αξιοπιστία και η λειτουργικότητα του κυκλώματος.

2.2 Διεπαφές Επικοινωνίας

Μεταξύ των components συνυπάρχουν 2 πρωτόκολλα επικοινωνίας:

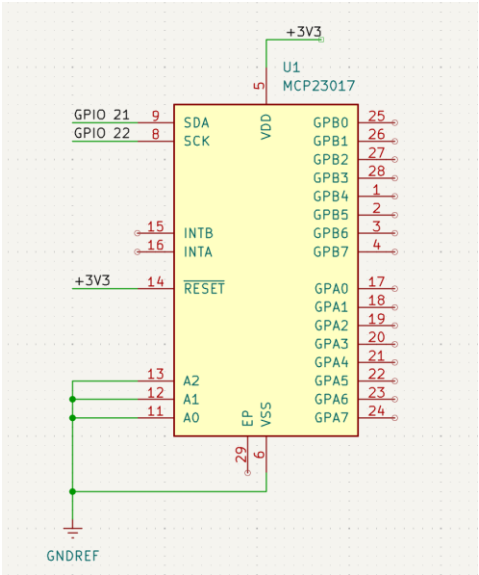
1. I2C Bus μεταξύ MCP23017 και ESP32 για έλεγχο των pins
2. SPI Bus μεταξύ LCD οθόνης και ESP32 για έλεγχο του GUI της οθόνης

Ακολουθεί μία σύντομη επεξήγηση της λειτουργίας των πρωτοκόλλων που αξιοποιήσαμε.

2.2.1 I2C Bus

Το I2C προσφέρει τη δυνατότητα ελέγχου και των 13 κουμπιών με μόνο 2 γραμμές σήματος: την SDA (Serial Data) και την SCL (Serial Clock). Με αυτόν τον τρόπο εξοικονομούνται 11 pins του ESP32 (2 pins αντί για 13), κάτι ιδιαίτερα πρακτικό για τη σχεδίαση του embedded συστήματος. Η επικοινωνία διεξάγεται στα 400kHz, συχνότητα η οποία εξασφαλίζει αμελητέο latency.

Ακολουθεί το κυκλωματικό διάγραμμα των συνδέσεων μεταξύ του MCP23017 και του ESP32:

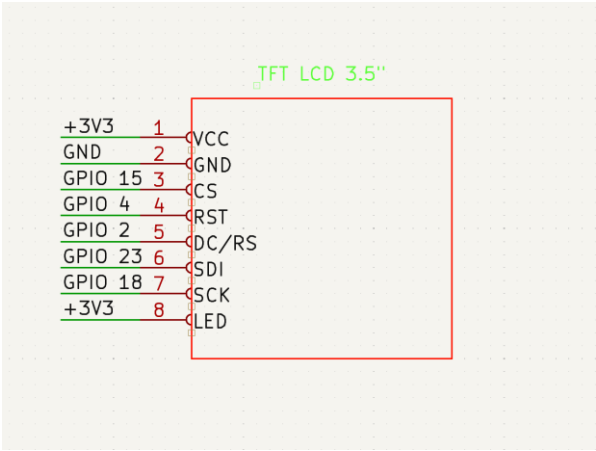


Συνδέσεις του MCP23017 στο ESP32 [2.2.1]

2.2.2 SPI Bus

Το πρωτόκολλο SPI χρησιμοποιείται για τη μεταφορά μεγάλου όγκου πληροφορίας στην οθόνη (480x320 pixel, αυτόνομα για κάθε pixel). Το SPI χρησιμοποιεί 4 γραμμές σήματος: SCK (Clock), MOSI (Master Out Slave In), CS (Chip Select) και DC (Data/Command). Η επικοινωνία είναι χρονισμένη στα 40MHz, μεγαλύτερη των 400kHz του I2C, κάτι απαραίτητο για να έχουμε real-time visual feedback και συνεπακόλουθα για ομαλή ανανέωση των γραφικών στην οθόνη. Η λειτουργία της οθόνης βασίζεται στον driver ILI9488.

Ακολουθεί το κυκλωματικό διάγραμμα της διασύνδεσης της οθόνης με το ESP32:



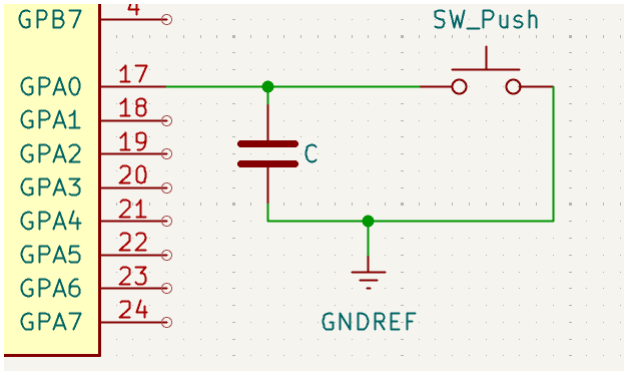
Συνδέσεις από την οθόνη στο ESP32 [2.2.2]

2.3 Διασύνδεση Εισόδων

Έχουμε 2 ειδών εισόδους: απλούς μηχανικούς διακόπτες τύπου button και ένα rotary encoder, οπότε η ανάλυσή μας θα χωριστεί σε δύο μέρη. Στη συνέχεια, θα αναλύσουμε την οργάνωση των κυκλωμάτων ανά breadboard.

2.3.1 Μηχανικοί Διακόπτες (Buttons)

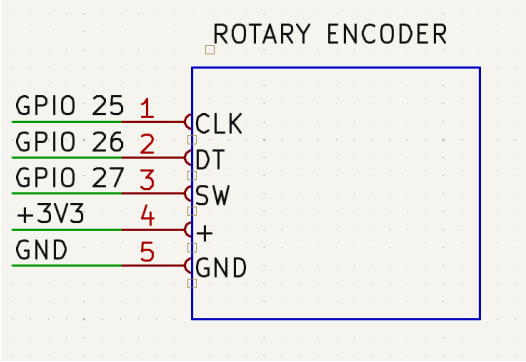
Η συσκευή περιλαμβάνει 13 μηχανικούς διακόπτες, οποίοι είναι συνδεδεμένοι στις GPA και GPB θύρες του MCP23017. Οι διακόπτες λειτουργούν με active-low λογική, δηλαδή η λειτουργία διεκπεραιώνεται στο λογικό '0'. Σε συνδυασμό με τους πυκνωτές, το σήμα καθαρίζεται με αποτέλεσμα να μην δημιουργούνται προβλήματα θορύβου.



Κύκλωμα με διακόπτη Button

2.3.2 Περιστροφικός Κωδικοποιητής (Rotary Encoder)

Για τον έλεγχο της έντασης (Volume Up/Down) και την έναρξη/παύση (Play/Pause) χρησιμοποιήθηκε ο rotary encoder. Η λειτουργία play/pause είναι εφάμιλλη με αυτή των buttons. Για τον περιστροφικό διακόπτη όμως έχουμε διαφορετική λειτουργία. Συγκεκριμένα, ο rotary encoder παράγει δύο τετραγωνικά σήματα DT (Data) και CLK (Clock) με διαφορά φάσης 90° . Ανάλογα με την κατεύθυνση της περιστροφής (δεξιόστροφη ή αριστερόστροφη) συγκρίνει πιο από τα δύο σήματα αλλάζει κατάσταση πρώτο και επιλέγει αντίστοιχα αύξηση ή μείωση της έντασης του ήχου. Τα pins του rotary encoder είναι απευθείας συνδεδεμένα πάνω στα pins του ESP32 και όχι μέσω του MCP23017, όπως παρακάτω:



Συνδέσεις Rotary Encoder σε ESP32

2.3.3 Breadboards

Στην συσκευή μας χρησιμοποιήσαμε 3 breadboards: 2 κλασσικού μεγέθους και ένα του μισού μεγέθους. Στο μικρό κεντρικό breadboard έχουν γίνει οι συνδέσεις του ESP32 με την οθόνη και με το rotary encoder. Στο πάνω κλασσικό breadboard έχουν τοποθετηθεί τα 10 από τα 13 buttons, ενώ στο κάτω έχουν τοποθετηθεί τα 3 επιπλέον buttons και η σύνδεση με το MCP23017.

3 C++ FIRMWARE & SERIAL BRIDGE

3.1 Δομή Firmware

Ο κώδικας του ESP32 έχει οργανωθεί ούτως ώστε να μην αποτελεί απλά ένα σύνολο εντολών, αλλά να είναι σχεδιασμένο να λειτουργεί ως ένα multitasking σύστημα. Έτσι, για παράδειγμα, η ανανέωση της οθόνης δεν εμποδίζει την λειτουργία των buttons ή του rotary encoder.

Για την υλοποίηση του firmware χρησιμοποιήθηκαν οι βιβλιοθήκες: Wire.h, Adafruit_MCP23X17.h και TFT_eSPI.h.

3.2 Διαχείριση εισόδων

Ο ESP32 ελέγχει συνεχώς στον MCP23017 την κατάσταση των buttons και αν κάποιο βρεθεί στο λογικό ‘0’, τότε ενεργοποιεί την αντίστοιχη λειτουργία. Όσον αφορά το rotary encoder, για την αποφυγή απώλειας δεδομένων κατά τη γρήγορη περιστροφή, χρησιμοποιήσαμε external interrupts στα pins του ESP32. Αυτό διασφαλίζει ότι η μεταβλητή της έντασης ενημερώνεται ακαριαία, ανεξάρτητα από το τι κάνει το υπόλοιπο πρόγραμμα.

3.3 Σχεδίαση GUI Οθόνης

Όπως αναφέραμε και στην παράγραφο 2.2.2, το SPI είναι ρυθμισμένο στα 40MHz που σε συνδυασμό με την βιβλιοθήκη TFT_eSPI επιτρέπει την ομαλή ανανέωση των pixel της οθόνης. Για να αποφευχθεί το flickering της οθόνης, δεν ανανεώνεται ολόκληρη η οθόνη συνεχώς, αλλά μόνο τα pixel τα οποία αλλάζουν.
Η οθόνη έχει χωριστεί σε 5 μέρη:

- PC Multimedia (Volume, Microphone On/Off, Camera On/Off)

- Spotify (Όνομα κομματιού – Καλλιτέχνης, progress bar που αναγράφει real-time τη χρονική στιγμή που βρίσκεται το κομμάτι)
- PC Stats (CPU - RAM - GPU usage)
- Discord (Mute/Unmute, Deafen/Undeafen)
- Gemini/Voice Command Status (Standby, Listening)

3.4 Σειριακό Πρωτόκολλο Επικοινωνίας

Σχεδιάσαμε ένα απλό πρωτόκολλο επικοινωνίας της μορφής <TAG:PAYLOAD>, στο οποίο το TAG προσδιορίζει την κατηγορία της εντολής και το PAYLOAD την μεταβληθείσα πληροφορία. Έτσι, με αυτό τον απλό τρόπο μπορεί και μεταβιβάζεται γρήγορα η πληροφορία από το ESP32 στο PC και αντίστροφα.

3.5 Λογική Serial Bridge

Η επικοινωνία ESP32 – PC είναι αμφίδρομη, οπότε η ανάλυση μας θα χωριστεί σε δύο διακριτά μέρη: από το microcontroller στο PC και από το PC στο microcontroller.

- ESP32 → PC: Μόλις ο ESP32 ανιχνεύσει κάποια μεταβολή στις εισόδους του κυκλώματος, σύμφωνα με το ενσωματωμένο firmware, οργανώνει την πληροφορία σύμφωνα με το επιλεγμένο σειριακό πρωτόκολλο επικοινωνίας και την προωθεί απευθείας στον υπολογιστή μέσω του καλωδίου USB.
- PC → ESP32: Ο ESP32 συνεχώς παρακολουθεί την σειριακή θύρα USB. Όταν εντοπίσει τον χαρακτήρα "<" διαβάζει το πακέτο πληροφορίας. Στη συνέχεια, δρομολογεί την εκάστοτε αλλαγή στη λειτουργία της συσκευής και ενημερώνει το GUI της οθόνης.

Επειδή η ταχύτητα της σειριακής είναι πολύ υψηλή (115200bps) και έχουμε μεγάλο όγκο δεδομένων, η ανάγνωση των δεδομένων γίνεται χαρακτήρα-χαρακτήρα, ώστε να μην μπλοκάρεται η λειτουργία της υπόλοιπης συσκευής. Επίσης, αν και τα δεδομένα από τις εισόδους αποστέλλονται στο PC σχεδόν ακαριαία, η ενημέρωση του GUI της οθόνης γίνεται ανά δευτερόλεπτο, ρυθμός ικανός να παρέχει μία ικανοποιητική εμπειρία χρήσης και συγχρόνως να μην υπερφορτώνει τον ESP32.

Για την αποφυγή σφαλμάτων, έχουμε δημιουργήσει δύο βασικούς μηχανισμούς ελέγχου:

1. Τα πακέτα πληροφορίας προς ανάγνωση θα πρέπει να αρχίζουν και να τελειώνουν με [< , >].
2. Αν το TAG δεν ανήκει σε ένα προκαθορισμένο κατάλογο εντολών, τότε η εντολή απορρίπτεται.

Τέλος, χρησιμοποιήθηκε λογική multithreading . Συγκεκριμένα, επειδή το voice recording είναι μία διεργασία η οποία δεσμεύει πολλούς πόρους από το ESP32 αυτό εκτελείται σε ένα ανεξάρτητο thread συνεχώς. Έτσι, αποφεύγονται οι διακοπές στην ηχογράφηση που προκαλούνται από τη συμφόρηση εντός του ESP32.

4 PYTHON SOFTWARE & API'S

4.1 Διαχείριση Λειτουργικού Συστήματος

Ο κώδικας της Python λαμβάνει συγκεκριμένα σήματα από το ESP32 βάσει του σειριακού πρωτοκόλλου επικοινωνίας και εκτελεί άμεσα τις κατάλληλες αλλαγές στο λειτουργικό σύστημα στον υπολογιστή. Συγκεκριμένα, έχουμε τις παρακάτω λειτουργίες:

- Έλεγχος ένταση ήχου: Αξιοποιούμε τη βιβλιοθήκη pycaw (Python Core Audio Windows Library), ώστε να έχουμε πρόσβαση στον ήχο των Windows. Η Python λαμβάνει εντολές από τον rotary encoder και βάσει του payload του πρωτοκόλλου ρυθμίζει το ύψος της έντασης.
- Διαχείριση μικροφώνου: Με τη χρήση του εξωτερικού προγράμματος NirCmd η Python αποκτά έλεγχο στη διαχείριση του global μικροφώνου των Windows.
- Διαχείριση κάμερας: Αξιοποιούμε τις βιβλιοθήκες pyautogui και τη ctypes που ενεργοποιεί ή απενεργοποιεί τη λειτουργία της κάμερας.
- Παρακολούθηση των πόρων: μέσω της βιβλιοθήκης psutil η Python αντλεί πληροφορίες σχετικά με την καταπόνηση του CPU και της RAM, ενώ μέσω της GPUtil την καταπόνηση της GPU.

4.2 Διαχείριση Εφαρμογών

Μέσω της Python μπορέσαμε να ενσωματώσουμε τις πιο σημαντικές εφαρμογές που χρησιμοποιεί στην ψηφιακή του καθημερινότητα ένας μηχανικός. Με την ενσωμάτωση αναφερόμαστε σε δημιουργία φυσικών κουμπιών στην συσκευή οι οποίες εκτελούν συντομεύσεις και συνεπικουρούν στην βελτίωση της παραγωγικότητας του χρήστη. Ακολουθεί ανάλυση των επιμέρους εφαρμογών.

4.2.1 Spotify

Για τον έλεγχο του Spotify αξιοποιήσαμε το Spotify Web API (το οποίο απολύτως δωρεάν) και τη βιβλιοθήκη spotipy, η οποία επιτρέπει την άντληση πληροφορίας για το όνομα του κομματιού και αναπαράγεται εκείνη τη στιγμή, το όνομα του καλλιτέχνη, την χρονική στιγμή που βρίσκεται το τραγούδι και τη συνολική διάρκεια.

Επιπλέον, υπάρχει η δυνατότητα play/pause του κομματιού, καθώς και previous/next για καλύτερο έλεγχο του Spotify. Η ενσωμάτωση αυτών των λειτουργιών στο φυσικό κουτί βελτιώνει σημαντικά την εμπειρία του χρήστη, ο οποίος επιθυμεί να συνδυάσει εργασία και μουσική και να έχει τον απόλυτο έλεγχο.

4.2.2 Discord (Καθολικός Έλεγχος Κώφωσης - Deafen)

Λόγω της αυστηρής απαγόρευσης των "Self-Bots" (έλεγχος προσωπικών λογαριασμών μέσω API) από το Discord, η ενσωμάτωση υλοποιήθηκε αναγκαστικά μέσω προσομοίωσης συντομεύσεων σε επίπεδο Λειτουργικού Συστήματος. Στον σχεδιασμό πάρθηκε η απόφαση να προγραμματιστεί αποκλειστικά η εντολή της Κώφωσης (Deafen: F14, εικονικό), καθώς εκμεταλλευόμενοι την εγγενή συμπεριφορά της πλατφόρμας, η Κώφωση επιβάλλει αυτόματα και Σίγαση (Mute). Αυτό εξοικονόμησε πολύτιμα GPIO pins, δημιουργώντας ένα "Panic Button" πλήρους αποκοπής. Επειδή η εντολή λειτουργεί ως διακόπτης εναλλαγής, αναπτύχθηκε μια εσωτερική μηχανή κατάστασης (discord_state = {"deaf": "OFF"}) ώστε η Python να καταγράφει το ιστορικό χρήσης.

4.2.3 Zoom (Raise Hand Integration)

Η αναζήτηση και χρήση της επιλογής "Σήκωμα Χεριού" (Raise Hand) μέσα στα μενού προκαλεί σημαντική απόσπαση προσοχής (Context Switching). Για την επίλυση αυτού του UX προβλήματος, αξιοποιήθηκαν οι καθολικές συντομεύσεις του Zoom. Όταν λαμβάνεται η εντολή ZOOM_HAND από το ESP32, η Python λειτουργεί ως γέφυρα και πυροδοτεί αόρατα τον συνδυασμό alt+y στο παρασκήνιο. Η επιλογή αυτή μετατρέπει μια πολύπλοκη διαδικασία λογισμικού σε μια άμεση, μηχανική (tactile) αλληλεπίδραση. Ο

χρήστης αποκτά ένα αποκλειστικό, φυσικό "Κουμπί Συμμετοχής", διεκδικώντας τον λόγο ακαριαία (zero-latency interaction), ενώ συνεχίζει να εργάζεται απρόσκοπτα σε άλλα προγράμματα, ακόμη και αν το Zoom είναι ελαχιστοποιημένο.

4.2.4 OBS (Έλεγχος μέσω OS-Level Hotkeys)

Η διαχείριση του OBS απαιτούσε άμεση απόκριση χωρίς να χάνεται η εστίαση του ενεργού παραθύρου (window focus). Για την αποφυγή διενέξεων με άλλες εφαρμογές (key conflicts) και τη μείωση της κατανάλωσης πόρων, υλοποιήθηκε μια ελαφριά αρχιτεκτονική OS-Level Injection. Μέσω της βιβλιοθήκης keyboard, η Python προσομοιώνει σύνθετες καθολικές συντομεύσεις (Global Hotkeys). Εφαρμόστηκε στρατηγική λογική "Toggle", χρησιμοποιώντας τον ίδιο ακριβώς συνδυασμό (Ctrl+Alt+7) για έναρξη και διακοπή καταγραφής, εξοικονομώντας πλήκτρα. Παράλληλα, ενσωματώθηκε η δυνατότητα άμεσης "τηλεοπτικής" εναλλαγής σκηνών (Ctrl+Alt+8 για "Screen" και Ctrl+Alt+9 για "BRB"). Η λύση αυτή εξασφαλίζει απόλυτη σταθερότητα και μηδενική καθυστέρηση (zero-latency), καθώς το OBS ανταποκρίνεται ακαριαία στο εικονικό πάτημα ακόμα και όταν είναι πλήρως ελαχιστοποιημένο.

4.3 Διεπαφή Χρήστη στον υπολογιστή (PC GUI)

Για την παραμετροποίηση του συστήματος από τον τελικό χρήστη, αναπτύχθηκε ένα παραθυρικό περιβάλλον (GUI) με τη χρήση της ενσωματωμένης βιβλιοθήκης tkinter. Το περιβάλλον δεν λειτουργεί απλώς ως οθόνη επιλογών, αλλά διαθέτει εξελιγμένη λογική διαχείρισης συστήματος.

4.3.1 Έξυπνη Ανίχνευση και Εκκίνηση Εφαρμογών

Το GUI έχει τη δυνατότητα να σαρώνει το σύστημα σε πραγματικό χρόνο. Χρησιμοποιώντας την κλήση συστήματος tasklist (μέσω της βιβλιοθήκης subprocess), ανιχνεύει αν εφαρμογές-κλειδιά όπως το Discord (discord.exe), το OBS Studio (obs64.exe) ή το Spotify εκτελούνται ήδη. Κατά το πάτημα του κουμπιού "Εκκίνηση", το πρόγραμμα δεν στέλνει απλά σήματα. Αναλαμβάνει να εκκινήσει αυτόματα όσες εφαρμογές έχει επιλέξει ο χρήστης, αξιοποιώντας τις προεπιλεγμένες διαδρομές (paths) των Windows. Ειδικά για προγράμματα όπως το OBS, το script φροντίζει να ορίσει το σωστό "Working Directory" (π.χ. C:\Program Files\obs-studio\bin\64bit), αποτρέποντας σφάλματα φόρτωσης βιβλιοθηκών (DLLs).

4.3.2 Δυναμική Βαθμονόμηση Μικροφώνου (Microphone Calibration)

Μία από τις μεγαλύτερες καινοτομίες στο κομμάτι του λογισμικού είναι το ενσωματωμένο σύστημα βαθμονόμησης περιβαλλοντικού θορύβου (Noise Calibration).

Η Python μέσω της βιβλιοθήκης pyaudio και του numpy, το σύστημα "ακούει" τον χώρο για 3 δευτερόλεπτα. Στη συνέχεια, τα ηχητικά δεδομένα μετατρέπονται σε πίνακες και υπολογίζεται η μέση ενέργεια του θορύβου (Root Mean Square - RMS). Βάσει του αποτελέσματος RMS, το πρόγραμμα αποφασίζει δυναμικά την ιδανική ευαισθησία. Χρησιμοποιώντας το εξωτερικό εργαλείο nircmd.exe, προσαρμόζει αυτόματα την πραγματική ένταση (Gain) του μικροφώνου στα Windows (π.χ. 95% για απόλυτη ησυχία, 50% για θορυβώδες περιβάλλον). Τα δεδομένα αυτά αποθηκεύονται σε αρχείο noise_profile.json για μελλοντική χρήση από αλγορίθμους φωνητικών εντολών.

4.4 Δυναμικά Προσαρμόσιμα Πλήκτρα (Custom Buttons)

Για να διασφαλιστεί η ευελιξία του Controller, το GUI ενσωματώνει σύστημα διαχείρισης 3 ελεύθερα προγραμματιζόμενων πλήκτρων (Custom Slots).

Οι ρυθμίσεις του χρήστη (όνομα πλήκτρου, διαδρομή, κατάσταση) αποθηκεύονται και ανακτώνται δυναμικά από το αρχείο custom_buttons.json. Το σύστημα (μέσω της συνάρτησης generate_name_from_content) αναλύει την είσοδο του χρήστη. Εάν η είσοδος είναι URL (π.χ. ξεκινά με http://), αναγνωρίζει αυτόματα γνωστές πλατφόρμες (YouTube, Reddit, Spotify) και ονομάζει το κουμπί ανάλογα. Κατά την εκτέλεση, το ανοίγει στον προεπιλεγμένο browser (μέσω webbrowser). Εάν είναι διαδρομή τοπικού αρχείου, εξάγει το όνομα του εκτελέσιμου και το τρέχει μέσω της εντολής os.startfile().

4.5 Ηχογράφηση και Επεξεργασία Φωνής

Για την λειτουργία της φωνητικής εντολής είναι απαραίτητη πρώτα η ηχογράφηση και η ψηφιακή επεξεργασία του σήματος. Επομένως, ας δούμε πιο αναλυτικά τις δύο διεργασίες.

4.5.1 Ηχογράφηση Φωνής

Με το παρατεταμένο πάτημα του κουμπιού της φωνητικής εντολής ενεργοποιείται το μικρόφωνο, έπειτα από επικοινωνία με το ESP32. Ο ήχος συλλέγεται σε chunks σε ξεχωριστό thread, ώστε να αποφεύγεται η παραμόρφωση (audio stuttering), και ακολούθως αποθηκεύεται σε αρχείο raw_command.wav. στον υποφάκελο recordings του φακέλου command.

4.5.2 Επεξεργασία Φωνής

Αρχικά, το πρώτο βήμα είναι ο καθαρισμός του θορύβου με τη χρήση της βιβλιοθήκης noisereduce. Το πρόγραμμα αναλύει το φάσμα συχνοτήτων του σήματος και αφαιρεί το στατικό θόρυβο ο οποίος υπάρχει. Έπειτα, μέσω της βιβλιοθήκης pydub κάνει κανονικοποίηση στο σήμα, ώστε τα ηχητικά των φωνητικών εντολών να έχουν όλα την ίδια ένταση άσχετα με την ένταση της ηχογράφησης και αποθηκεύεται σε αρχείο clean_command.wav στον υποφάκελο processed του commands.

4.5.3 Speech-to-text μέσω της Google

Για να μπορέσει το σύστημα να ανταποκριθεί σε φωνητική εντολή, αυτή θα πρέπει να έχει γίνει κείμενο. Επομένως, αξιοποιήσαμε την βιβλιοθήκη speech_recognition, η οποία είναι από τις καλύτερες (ειδικά στα ελληνικά) λόγω της πιστότητας της. Έτσι, έχουμε την εντολή σε ένα string το οποίο στη συνέχεια είναι πολύ πιο ευέλικτο στη διαχείριση.

4.6 Απόκριση σε Φωνητικές Εντολές και Ενσωμάτωση AI

Για την διαχείριση των φωνητικών αιτημάτων έχουμε σχεδιάσει δύο τρόπους επεξεργασίας: τοπική ανάλυση (Local Parsing) ή ευφυής ανάλυση (AI Parsing). Παρακάτω θα δούμε αναλυτικότερα τι προσφέρει κάθε είδος επεξεργασίας. Πριν όμως το στάδιο εκείνο το στάδιο πραγματοποιείται κανονικοποίηση στο string της εντολής, δηλαδή αφαιρούνται οι τόνοι και όλα τα γράμματα γίνονται πεζά, ώστε να αποφεύγονται σφάλματα λόγω διαφορετικού τρόπου γραφής.

4.6.1 Local Parsing

Είναι η ταχύτερη μέθοδος επεξεργασίας που διαθέτει το σύστημα. Ουσιαστικά, το σύστημα αναζητάει ρίζες λέξεων (stemming) μέσα στο string (πχ «δυναμ» αντί για δυνάμωσε, «χαμηλ» αντί για χαμήλωσε) και ανάλογα με το τι θα εντοπίσει η Python, πυροδοτεί την αντίστοιχη μεταβολή και ενημερώνει τον ESP32. Ο λόγος που αναζητούμε ρίζες λέξεων και όχι ολόκληρες λέξεις είναι διότι είναι πολύ συχνό να μην αναγνωρίζονται ακριβώς οι λέξεις, αλλά να διαφοροποιείται το κείμενο που προέρχεται από την ίδια εντολή, ανάλογα με την άρθρωση του χρήστη.

Το πλεονέκτημα αυτής της μεθόδου είναι η μεγάλη ταχύτητα (μικρότερος χρόνος απόκρισης από 0.5 sec), κάτι ιδιαίτερα χρήσιμο για απλές σύντομες εντολές (έως 9 λέξεις). Βέβαια, αν έχουμε δύο ή σε κάποιες περιπτώσεις τρεις υποεντολές σε ένα κείμενο το πολύ 9 λέξεων όπως πχ «Χαμήλωσε την ένταση στο 40 και ξεκίνα το Spotify», ανάλυση πάλι γίνεται τοπικά και ικανοποιούνται και τα δύο αιτήματα. Έτσι, δεν απαιτείται σύνδεση στο διαδίκτυο για απλές εντολές του τύπου «Βάλε την ένταση στο 80» ή «Άνοιξε το μικρόφωνο/κάμερα» και η εκτέλεσή τους είναι ταχύτατη.

4.6.2 AI Processing

Εάν το string της εντολής αριθμεί από 10 λέξεις και πάνω ή δεν αναγνωριστεί κάποια ρίζα λέξης μέσω stemming και οπότε απαιτείται κατανόηση του context της εντολής, το string αποστέλλεται στο Gemini AI μέσω του ομώνυμου API. Για λόγους αξιοπιστίας, δημιουργήσαμε δύο client keys (ένα κύριο, ένα backup) ώστε να αποφεύγονται τα σφάλματα.

Το Gemini λαμβάνει ένα prompt που του εξηγεί το ρόλο του ως ένα «ευφυές» All-in-one Controller, αναγράφει κατά λέξη την εντολή και του ζητάει να απαντήσει τυποποιημένα σε μορφή JSON. Η Python έχει ένα λεξικό με τα αντικείμενα που μας ενδιαφέρουν (Spotify status, Volume, Microphone, κλπ.) σε μορφή JSON, κάτι που βοηθάει στην οργάνωση και στην επικοινωνία με το serial bridge, ειδικά σε σύνθετες εντολές.

Το πλεονέκτημα αυτής της μεθόδου είναι οι δυνατότητες όχι μόνο γλωσσικής κατανόησης, αλλά συνολικής κατανόησης πρόθεσης. Δηλαδή, με την εντολή «Δεν ακούω τίποτα», το Gemini AI αντιλαμβάνεται το πρόβλημα (χαμηλή ένταση ήχου) και δίνει τη λύση (ανεβάζει την ένταση του ήχου σημαντικά). Επειδή η απόκρισή του απαιτεί περίπου 1 δευτερόλεπτο, χρόνος διπλάσιος από τον χρόνο του local parsing, επιλέγεται η χρήση του ανάλογα με την πολυπλοκότητα της εντολής.

5 3D PRINTING & DESIGN

5.1 Φιλοσοφία και Διαδικασία Σχεδίασης

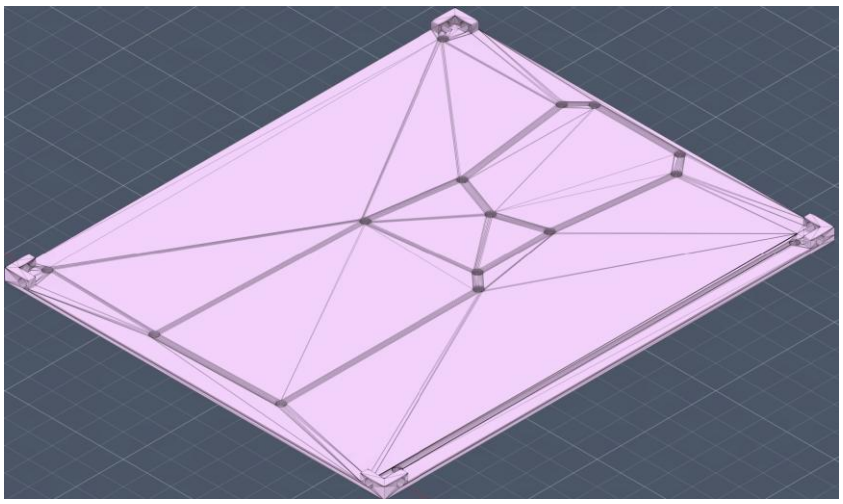
Η σχεδίαση του περιβλήματος (κουτιού) πραγματοποιήθηκε με κύρια προτεραιότητα η συσκευή να είναι πλήρως modular, δηλαδή χωρισμένη σε πολλά κομμάτια τα οποία συνδέονται μεταξύ τους για να παράγουν το τελικό αποτέλεσμα. Αυτό έγινε για λόγους ευκολίας τόσο στην διαδικασία της εκτύπωσης , καθώς έτσι το κάθε λάθος θα μπορούσε να διορθωθεί εύκολα και γρήγορα, όσο και για εύκολη πρόσβαση στα εσωτερικά της συσκευής κατά την διαδικασία συναρμολόγησης. Επιπλέον αυτό βοήθησε να αποφευχθεί σπατάλη υλικού στην διαδικασία εκτύπωσής καθώς ένα λάθος δεν θα σήμαινε ότι θα έπρεπε να ξεκινήσει η διαδικασία εκτύπωσης ξανά από την αρχή.

Τα επιμέρους κομμάτια σχεδιάστηκαν εξ ολοκλήρου στο πρόγραμμα “Autodesk Fusion” εξαιτίας της ευκολίας που προσφέρει στην δημιουργία λεπτομερών μοντέλων. Η εκτύπωση έγινε σε εκτυπωτή “Ender 3 S1” χρησιμοποιώντας το πλαστικό υλικό PLA. Οι ρυθμίσεις που χρησιμοποιήθηκαν ήταν 20% infill με initial layer speed 0.10 mm/sec. Κατά την κατασκευή των τμημάτων δόθηκε ιδιαίτερη έμφαση στην ποιότητα και στην ακρίβεια διότι με τον τρόπο αυτό θα εξασφαλίζαμε ένα περίβλημα το οποίο δεν θα είχε προβλήματα όχι μόνο στην εμφάνιση αλλά και στην συναρμολόγηση.

5.2 Ανατομία Περιβλήματος

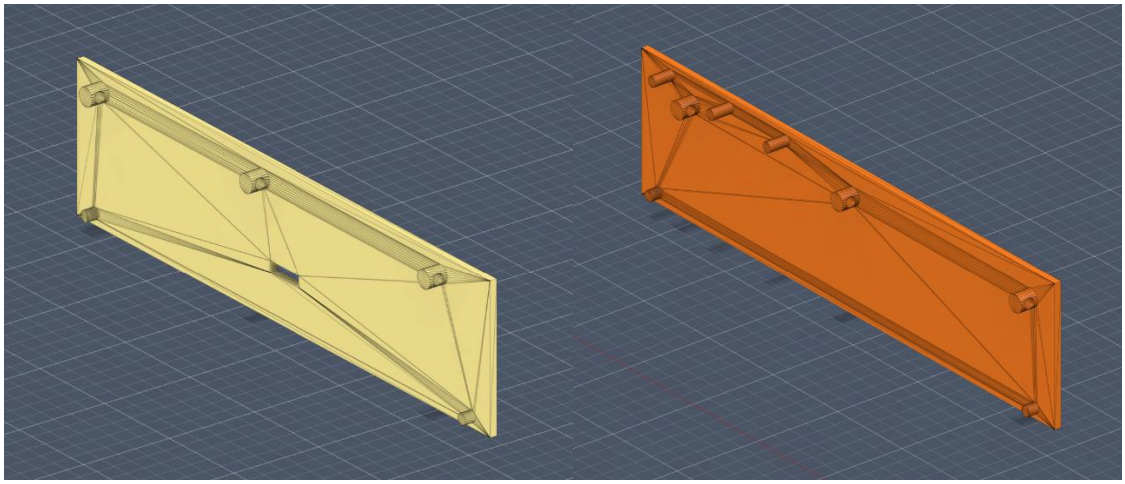
Το περίβλημα χωρίστηκε σε βάση (Base Plate , [5.2.1]), 4 μεταξύ τους ιδιαίτερες πλευρές, το άνω κάλυμμα και τα στηρίγματα της οθόνης και του Esp32.

Η βάση είναι ένα ορθογώνιο παραλληλόγραμμο με διαστάσεις 18cm x 16.5cm x 4cm η οποία έχει ως κύρια χαρακτηριστικά τα ανοίγματα τα οποία λειτουργούν ως ειδικές εγκοπές των βάσεων στήριξης των υπολοίπων εξαρτημάτων.



Βάση [5.2.1]

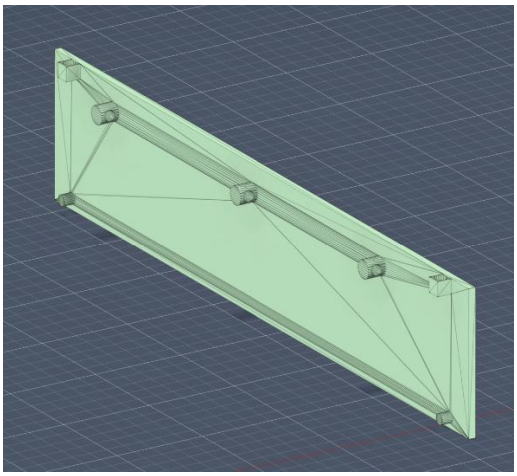
Οι τέσσερις πλευρές είναι σχεδιασμένες ξεχωριστά διότι η κάθε μια έχει να εξυπηρετήσει διαφορετικές λειτουργίες. Οι δυο πιο μικρές πλευρές, οι οποίες φαίνονται από κάτω στο εξής θα αναφέρονται ως «Πλευρά Α» [5.2.1] και «Πλευρά Β» [5.2.2] έχουν τις μεγαλύτερες διαφορές.



Πλευρά Α [5.2.2] *Πλευρά Β [5.2.3]*

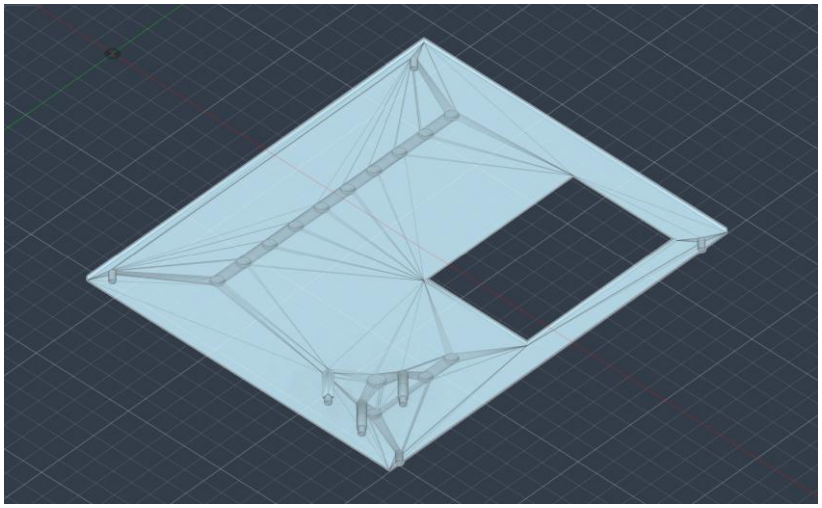
Η πλευρά Α έχει τον ρόλο να στηρίζει τη βάση της οθόνης. Με αυτό υπόψιν ενσωματώθηκαν 3 στηρίγματα τα οποία κουμπώνουν πάνω στη βάση της οθόνης. Η πλευρά Β διαθέτει οπή ώστε να περνάει το καλώδιο που συνδέεται με τον υπολογιστή.

Οι δύο επιμήκεις πλευρές [5.2.3] είναι πανομοιότυπες χωρίς κάποια ιδιαίτερα χαρακτηριστικά πέρα από τις εξοχές που φέρουν για να συνδέονται με τη βάση (Base Plate) και 2 εσοχές οι οποίες συνδέουν τις πλευρές αυτές με το άνω κάλυμμα.



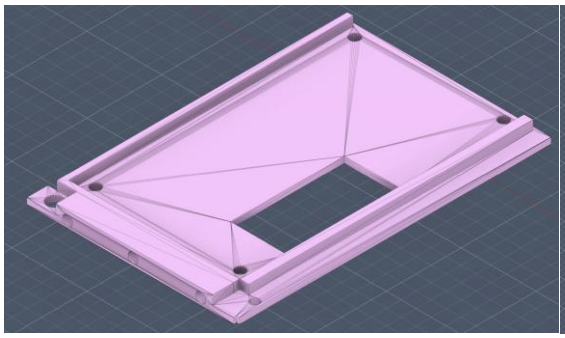
Επιμήκης πλευρά [5.2.4]

Το άνω κάλυμμα [5.2.5] έχει 14 μικρά ανοίγματα για τα καλύμματα των κουμπιών (Keycaps) και ένα μεγαλύτερο για την οθόνη καθώς και ένα για το ενσωματωμένο rotary encoder. Επιπλέον διαθέτει 4 εξοχές οι οποίες συνδέονται με αντίστοιχες εσοχές στις μακρινές πλευρές.

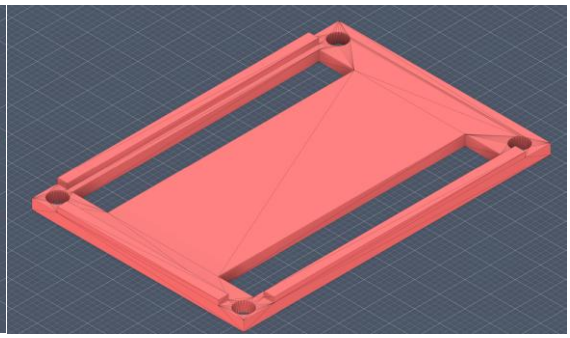


Άνω κάλυμμα [5.2.5]

Τα στηρίγματα της οθόνης [5.2.6] και του ESP32 [5.2.7] ήταν τα πιο δύσκολα στη σχεδίαση, καθώς απαιτούσαν υψηλή ακρίβεια στους υπολογισμούς των διαστάσεων τους και προσοχή ώστε να σχεδιαστούν με τρόπο που δεν θα επιφέρει ζημιά στα ηλεκτρονικά αλλά ούτε θα εμποδίσει την λειτουργία τους. Επιπλέον για κάθε ένα από τα στηρίγματα απαιτούνταν περαιτέρω κυλινδρικοί αποστάτες (standoffs) ώστε η κατασκευή να είναι πιο στιβαρή.

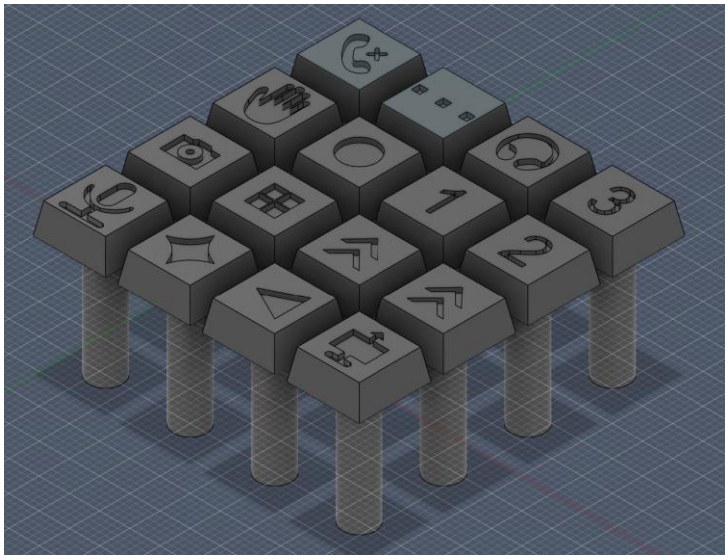


Στήριγμα Οθόνης [5.2.6]



Στήριγμα ESP32 [5.2.7]

Τα καλύμματα των κουμπιών (keycaps) [5.2.8] σχεδιάστηκαν ξεχωριστά για κάθε κουμπί, ώστε ο χρήστης να αναγνωρίζει άμεσα τη λειτουργία του. Αυτό επιτεύχθηκε μέσω της χάραξης ειδικών συμβόλων στην επιφάνειά τους.



Καλύμματα κουμπιών (Keycaps) [5.2.8]

5.3 Ανοχές και Διορθώσεις κατά τη Συναρμολόγηση

Κατά τη σχεδίαση των επιμέρους τμημάτων λήφθηκε υπόψιν η συμπεριφορά του πλαστικού υλικού PLA, το οποίο έχει την τάση να συρρικνώνεται ελαφρώς μετά την εκτύπωση. Για να μπορούν να ενώνονται τα κομμάτια σωστά μεταξύ τους, εφαρμόστηκαν συγκεκριμένα κενά (ανοχές) στα σημεία εφαρμογής. Συγκεκριμένα, οι τρύπες στις οποίες κουμπώνουν τα εξαρτήματα σχεδιάστηκαν με διάμετρο 0.335cm, ενώ οι προεξοχές που μπαίνουν μέσα σε αυτές σχεδιάστηκαν στα 0.295cm. Αυτό έγινε με κύρια προτεραιότητα να εξασφαλίσουμε ένα καλό κούμπωμα, χωρίς να ασκείται υπερβολική πίεση στο πλαστικό που θα μπορούσε να το σπάσει κατά τη συναρμολόγηση.

Παρόλα αυτά, κατά τη διαδικασία της εκτύπωσης υπήρξαν περιπτώσεις όπου τα κομμάτια δεν εφάρμοζαν τέλεια μεταξύ τους λόγω μικρών ατελειών του 3D εκτυπωτή. Για λόγους ευκολίας και για να αποφευχθεί η σπατάλη χρόνου και υλικού, μια τέτοια αστοχία δεν σήμαινε ότι θα έπρεπε να ξεκινήσει η εκτύπωση ξανά από την αρχή. Αντίθετα, τυχόν λάθη στις διαστάσεις τα διορθώναμε με το χέρι χρησιμοποιώντας λίμα (τρύφτη). Αφαιρώντας τοπικά το πλαστικό που περίσσευε, καταφέραμε να φέρουμε τα εξαρτήματα στην κατάλληλη μορφή ώστε να κουμπώσουν ακριβώς όπως είχαν σχεδιαστεί.

6 CONCLUSION

Έχοντας παρουσιάσει το σύνολο της συσκευής μπορούμε να εξάγουμε κάποια συμπεράσματα. Πρώτα απ' όλα, όσον αφορά το κόστος, αυτό παρατηρούμε ότι ήταν εφικτό να παραμείνει κοντά στα 60€ λόγω της χαμηλής τιμής του ESP32 και του MCP23017, χωρίς όμως να υπάρξει περιορισμός της υπολογιστικής ισχύος. Επιπλέον, η συσκευή μας με την ενσωμάτωση της οθόνης 3.5" που ενημερώνει ζωντανά για τη λειτουργία του υπολογιστή και του AI Voice Assistant καινοτομεί στην αγορά των All-in-one Controllers, επιτυγχάνοντας να διατηρήσει υψηλή λειτουργικότητα, αξιοπιστία και χαμηλό κόστος. Ακόμη, η χρήση PLA και 3D printing,

καθώς και η modular σχεδίαση του κελύφους, δίνουν στον χρήστη την ευελιξία της αναβάθμισης και της επισκευής, αλλά και το προνόμιο της μοναδικότητας μέσω της εξατομίκευσης κάθε συσκευής.

7 ΒΙΒΛΙΟΓΡΑΦΙΑ – ΣΥΝΔΕΣΜΟΙ

7.1 Πηγές

1. Hardware & Datasheets (Μικροελεγκτές & ICs)

1. **Espressif Systems.** ESP32 Series Datasheet. Link
2. **Microchip.** MCP23017 16-Bit I/O Expander with I2C Interface. Link
3. **Texas Instruments.** Understanding the I2C Bus. Link
4. **Analog Devices.** SPI Interface Block Guide. Link

2. Arduino Libraries & Firmware (C++)

7. **Bodmer.** TFT_eSPI Library for ESP32/ESP8266. Link
8. **Adafruit.** MCP23017 Arduino Library. Link
9. **Espressif.** ESP-IDF Programming Guide (I2C Driver). Link

3. Python Backend & Automation APIs

11. **Python Software Foundation.** Python 3.12 Documentation. Link
12. **Gough, A.** Pycaw: Python Core Audio Windows Library. Link
13. **Giampaolo, R.** psutil: Process and system utilities. Link
14. **Sweigart, A.** PyAutoGUI Documentation. Link
15. **Spotipy.** Lightweight Python library for the Spotify Web API. Link

4. Digital Signal Processing & AI (NLP)

16. **Google AI.** Gemini API Documentation & Quickstart. Link
17. **Sainburg, T.** Noisereduce: Noise reduction in Python. Link
18. **SpeechRecognition.** Library for performing speech recognition. Link
19. **Pydub.** Audio manipulation with Python. Link

7.2 Σύνδεσμοι

GitHub project link: https://github.com/istratoudakis/-.git